you've got something you think should be there, drop me a line. When building Django projects for clients, I frequently found that there were certain kinds of code I was writing over and over, so eventually, I stuck it all in a library, with the result being Awl. Features found here include the following:

- Models
- Admin tools
- View decorators
- Context processors
- Commands
- Template tags and filters
- Testing utilities

You install Awl by running `python -m pip install django-awl` and adding `"awl"` to `INSTALLED_APPS`. As with Django Extensions, there is too much here to go through it all, but I'll describe a few things I use most often. There is an abstract `Model` you can inherit that adds create and update timestamp fields, `Model` classes for counters and locks, and a specialty `Model` for including a rank number for situations when your objects need an order. The ranked `Model` includes plugins for the Django Admin, providing you with up and down buttons to change an object's rank.

Often, when I'm playing around in Python, I create a quick script to test out an algorithm or double-check that weird syntax thing I just can't recall. The REPL is good for this, but it doesn't have the friendliest interface for typing a lot of code, so a quick script is often the answer. With Django, that's problematic. You can't just run a script within a Django context—you have to complete extra setup for the script to work. Enter the `run_script` command. It takes a Python file as an argument and loads and runs it, but within a Django context. This can be faster than writing a custom Django command and useful if you need a quick one-off.

Writing `ModelAdmin` classes can require a fair amount of boilerplate code, especially if you want to include cross-object links, like you were shown in chapter 5. The `awl.admintools.FancyModelAdmin` is a factory for `ModelAdmin` classes that supports additional features for the `list_display`. With a single line of code, you can add columns that have a default value when empty, links to related objects, and columns populated based on a template.

The feature I use most in Awl is a module called *Waelsteng*, an Anglo-Saxon word for *spear*. This module is a helper for testing; it includes tools for testing your Django Admin code and an object for faking requests if you want to call a view directly. It also has a custom test runner that supports managing dummy directories for uploads and, my personal favorite, a shortcut for running a specific test. The default Django test runner allows you to run a test with its full module name—for example, `python manage.py test bands.tests.TestBands.test_musician_view`. That's a lot of typing. Awl's replacement runner supports shortcutting with an equal sign (`=`). The same test could be run with `python manage.py test =musician_view`.